`jcmd` commands for the heap

Appendix

Command	Shows
`GC.class_histogram`	Instances and bytes per class
`GC.heap_info`	Heap regions: total, committed and used
`GC.heap_dump <file>`	HPROF dump for a heap analyzer
`GC.run`	Requests a full GC
`VM.flags`	The JVM flags in effect
`VM.native_memory`	Native memory by area (needs `-XX:NativeMemoryTracking`)
`Thread.print`	Thread dump

Watching garbage with `jstat -gc`

Appendix

`-gc` shows the same counters in KB, so eden size and GC count give a rough allocation rate.

```
$ jstat -gc <pid> 1s 3                    (columns trimmed, sizes in KB)
       EC           EU          OU      YGC     YGCT
 124928.0     35840.0      2180.4      14    0.120
 125952.0    115712.0      2180.4      15    0.125
 130048.0     61440.0      7300.4      17    0.135
```

- EU climbs towards EC, a young GC runs, and eden starts again: a sawtooth.
- Eden capacity times young GCs per second is roughly the allocation rate: about 125 MB × 1.5/s ≈ 190 MB/s.
- `jstat -gccause` adds the cause of the last GC, here `G1 Evacuation Pause`.

Choosing a `jstat` option

Appendix

Option	Shows
`-gcutil`	Percent used per generation, GC counts and times
`-gc`	The same in KB: capacities and used sizes
`-gccause`	`-gcutil` plus the cause of the last and the current GC
`-gccapacity`	Minimum, maximum and current capacity per generation
`-gcnew`	Young generation: survivor spaces and tenuring
`-class`	Classes loaded and unloaded

JOL: one `Integer`, compact headers on

Appendix

With `-XX:+UseCompactObjectHeaders` the class pointer moves into the 8 B mark word.

```
java.lang.Integer object internals:
OFF  SZ   TYPE DESCRIPTION               VALUE
  0   8        (object header: mark)     0x001ac00000000001 (Lilliput)
  8   4    int Integer.value             500
 12   4        (object alignment gap)
Instance size: 16 bytes
Space losses: 0 bytes internal + 4 bytes external = 4 bytes total
```

- The header shrinks from 12 B to 8 B, but the object still rounds up to 16 B.
- The 4 B saved becomes an alignment gap: a box is a box.

JOL: the footprint of a small list

Appendix

```
List<Integer> list = new ArrayList<>(List.of(500, 501, 502));
System.out.println(GraphLayout.parseInstance(list).toFootprint());
```

Object	Count	Headers off	Headers on
`Object[]` (the backing array)	1	32 B	24 B
`Integer`	3	48 B	48 B
`ArrayList`	1	24 B	24 B
Total	5	104 B	96 B

- `GraphLayout` follows every reference, so the boxes count too: half the bytes are `Integer`.

Adding JOL to a project

Appendix

```
<dependency>
    <groupId>org.openjdk.jol</groupId>
    <artifactId>jol-core</artifactId>
    <version>0.17</version>
</dependency>
```

```
java -XX:+EnableDynamicAgentLoading -Djdk.attach.allowAttachSelf=true ...   # let JOL attach to its own JVM
java -jar jol-cli.jar internals java.util.HashMap                            # no code: the command line tool
```

- `jol-cli` is built from the openjdk/jol repository; it also has `footprint` and `heapdump-*` modes.
- jol-samples in the same repository walk through headers, alignment, arrays and more.